AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Lesson 4

Plan Mode and Iterative Refinement

An hour of confident edits, and then the realization that the wrong service had been changed. Claude stops when work looks done, so give it a way to know: plan when the approach is unclear, and hand it a check it can run.

Module	AI12 of 13
Lesson	4 of 5
Prerequisites	AI12 L1–L3, AI06 L7
Exam relevance	**Core: TS 3.4, 3.5.** Also TS 1.7, 5.4
Reading time	~30 minutes

Contents

1. Plan Mode and Iterative Refinement

Lesson 4 of 5 · Claude Code

Plan Mode and Iterative Refinement

Key idea

Plan when the approach is uncertain, and execute directly when it's clear. Then steer with evidence: 2-3 input/output examples, tests that fail, or an interview that surfaces what you didn't think of. Send interacting issues together, and independent ones in sequence. Manage the session itself, since a stale or crowded context gives wrong answers with confidence.

> *Before: built-in tools and subagents. Teaches: plan mode, Explore, input/output examples, test-driven iteration, the interview pattern, batching feedback, verification, and session management. Used by: L5, AI13.*

The hook: the cost of the wrong start

An invented example, with a very common shape. A developer types: "Move auth to the new library." Claude starts editing immediately. Forty files later, it turns out that one service signs tokens differently, the new library can't do that, and the approach is wrong. The developer rewinds an hour of work.

Nothing was wrong with the edits. The approach was never examined. A ten-minute plan would have found that service, or at least asked about it.

The opposite mistake is also real: a plan-mode session and a review, for a one-line fix with a clear stack trace.

Intuition: building work

You don't draw up plans to change a light bulb, and you don't start knocking down walls without knowing which ones hold up the roof. The question is how much it would cost to be wrong, and how much you know about what's behind the wall.

Mechanism

Plan or direct?

Choose plan mode when	Choose direct execution when
Large-scale changes, or many files	One clear, small change
Several valid approaches	The fix is known
Architectural decisions	A clear stack trace in one file
An unfamiliar area	A well-understood area
Being wrong would be costly	Being wrong is cheap to fix

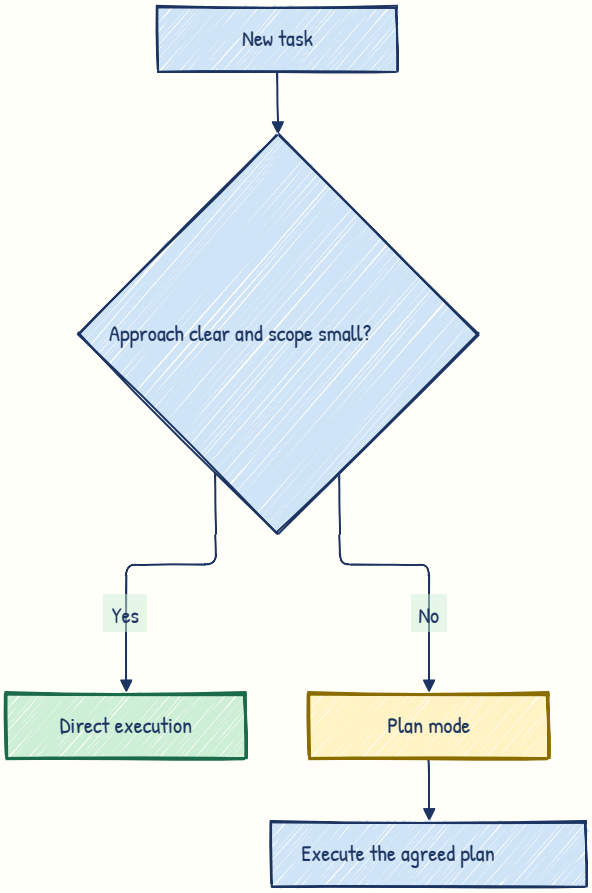


Figure 1. Uncertainty and the cost of being wrong decide it, rather than how important the task feels.

How plan mode works, from the docs:

Step	What happens
Enter	`Shift+Tab`, `/plan`, or `--permission-mode plan`
Research	"Claude reads files, runs shell commands to explore, and writes a plan, but does not edit your source."
Present	It shows the plan and asks how to proceed
Decide	Options include approving with auto mode, approving with manual edits, or continuing to plan
Execute	Approving exits plan mode, into the mode you chose

Plan, then execute is the guide's combination: investigate in plan mode, and carry out the agreed approach directly. Staying in plan mode forever isn't the idea.

Task changes. If a "simple" task turns out to touch a shared abstraction, switch to plan mode. Mode should follow what you've learned.

The Explore subagent

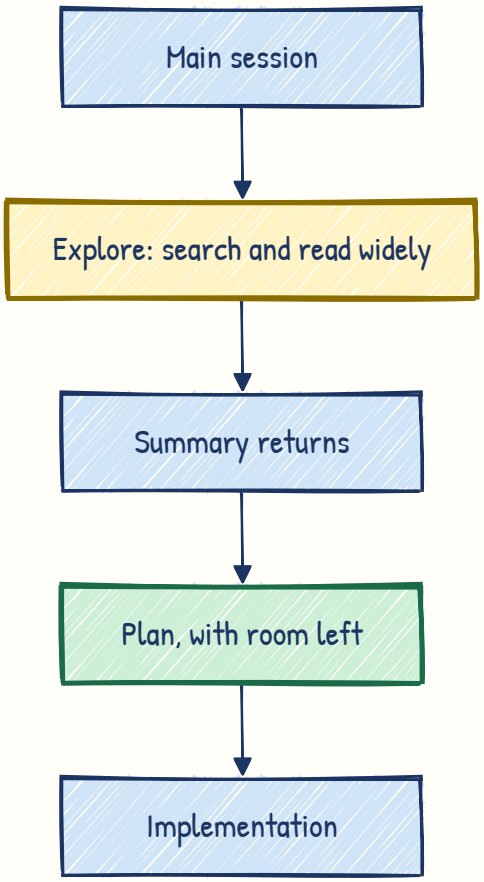


Figure 2. Verbose discovery stays in Explore. The main session keeps the plan, and has room left for implementation.

Explore is read-only, with Write and Edit denied. Per the subagents page, it and Plan skip CLAUDE.md and git status at startup. The guide's reason for it is that a multi-phase task can exhaust its context on discovery before implementation begins. The current best practices page says that running out of context is rarely the problem now, while subagents are still useful for keeping noisy output out of the way.

Steering with evidence

When output isn't right, more adjectives won't help. These are what work, per the guide:

Technique	Use when	Example
2–3 input/output examples	A transformation's wording is read inconsistently	`"3 days ago"` → `2026-11-03`
Tests first	Behavior can be specified	Share failures to guide each round
Interview pattern	You don't know what you don't know	"Ask me about what I've missed before you start"
A specific test case	One edge case fails	Null value, with the expected output

The interview pattern. The best practices page suggests a prompt such as "Interview me until you could write the spec, then write it to SPEC.md." Claude asks about what you haven't pinned down, and the guide's examples are cache invalidation and failure modes. The page then advises starting a fresh session to build from that spec, with a clean context and a written definition of done. It suits unfamiliar areas, and a well-specified change doesn't need it.

Together or in sequence?

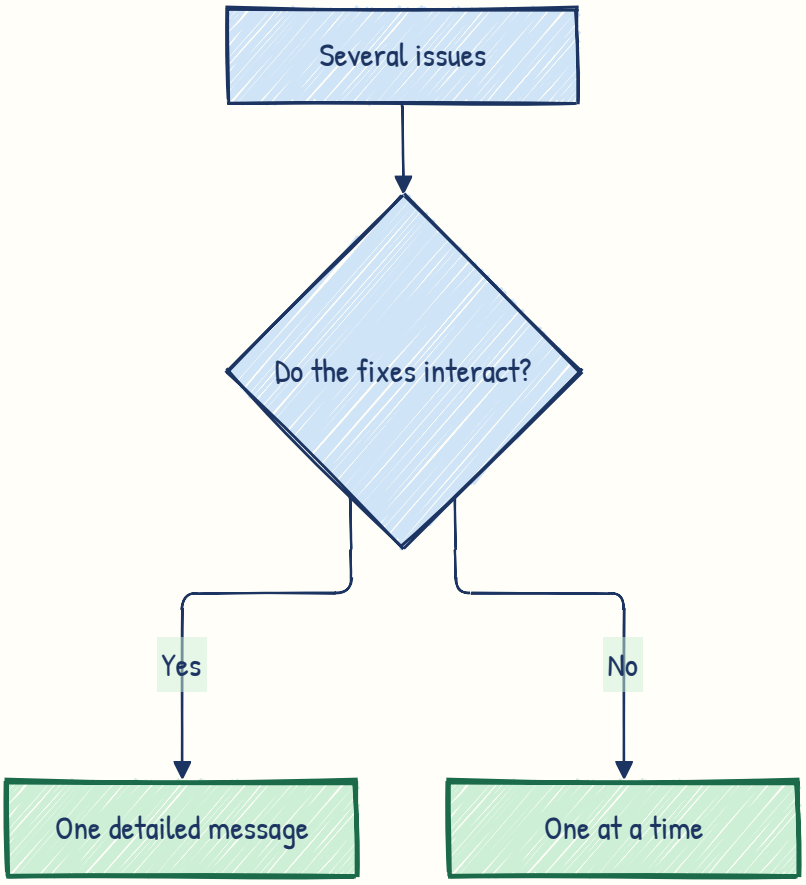


Figure 3. Interaction decides it. If fixing one changes another, Claude has to see them together.

Validation, error messages and logging in one flow interact, and fixing them one by one causes regressions. Three bugs in three unrelated modules can be handled in sequence.

Give it a check to run

The best practices page puts this first: "Claude stops when the work looks done." If it has something that comes back pass or fail, it can close the loop itself.

Level	How	Who decides it's done
In the prompt	"Run the auth tests, and keep going until they pass"	Claude
As a goal	`/goal all tests in tests/auth pass`. A separate model checks	A checker model
As a hook	A `Stop` hook runs the check	The harness
In CI	Branch protection	The pipeline

If your repo has one such command, it's the most useful line in CLAUDE.md.

Managing the session

AI06 L7 covered sessions. Claude Code's commands, from the sessions page:

Command	Does
`claude --continue`	Reopens the most recent session in this directory
`claude --resume <name>`	Resumes a named session
`claude -n auth-refactor`	Names it at startup. `/rename` works during a session
`/branch [name]`	Copies the conversation, and switches into the copy
`--continue --fork-session`	The same from the command line
`/clear`	Starts with an empty context, and saves the old session
`/compact [focus]`	Replaces history with a summary
`/context`	Shows what's using context

Choosing, per TS 1.7:

Situation	Do this
Earlier context is valid, and a few files changed	Resume, and name the changed files
Most results are now stale	New session, with a structured summary
Trying two approaches	Fork or branch from the analysis
Context full of verbose search output	`/compact`, or delegate next time
Moving to an unrelated task	`/clear`

Forking branches the conversation, and not the files. AI06 L7 made this point: two forks editing one directory will collide. Use worktrees to separate files.

The confirmed flags. The guide's `fork_session` is the SDK's option. On the CLI, it's `--fork-session`, and I confirmed `--continue` and `--fork-session` on the sessions page and in `claude --help` here.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Always plan	Wastes time on clear fixes
Never plan	Costly rework on large changes
"Try harder" feedback	Gives no information
Longer prose description	Keeps the ambiguity
One fix at a time, always	Interacting fixes regress
Trust "done"	Needs a check it can run
Resume, and say nothing	Stale results give confident errors

How Anthropic solved it

Mechanism	What's published	Source
Plan mode	Read and plan, with no source edits	Permission modes page
Explore	Read-only subagent	Subagents page
Verification first	"Give Claude a check to run"	Best practices
Interview	Have Claude ask questions first	Best practices
Sessions	Resume, branch, compact	Sessions page
Rewind	Restore code and conversation	Checkpointing page

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a single long session, with verbal feedback.

Improved: plan first and tests. New problem: long tasks degrade as context fills, and "done" is still Claude's own judgment.

Production: a workflow where each phase has its own context and a check.

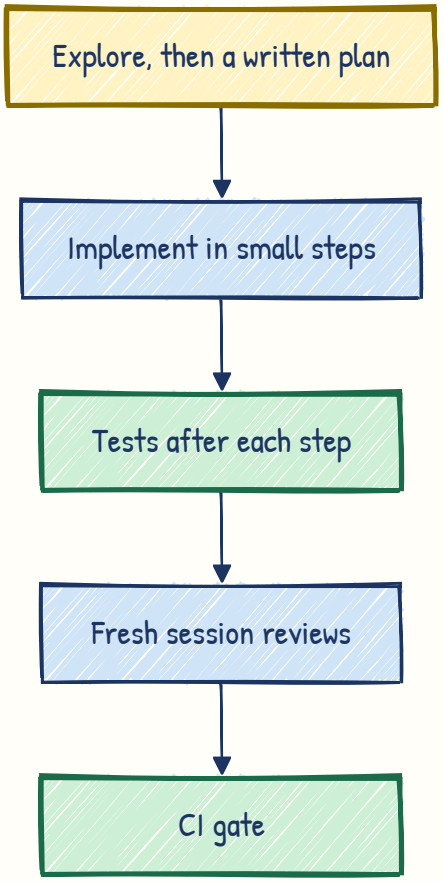


Figure 4. Each phase hands a written artifact to the next. The check decides what counts as done.

Component	Why it exists	How it communicates
Written plan	Survives context resets	A file
Tests	Pass-or-fail signal	Commands in CLAUDE.md
Small commits	Limit how far a mistake reaches	Git
Fresh review session	No generator's reasoning (TS 4.6)	Diff and criteria
Named sessions	Return to investigations	`--resume`
CI	The last word	Branch protection

Failures:

Failure	How you'd notice	What to do
Wrong approach	Rework	Plan earlier, and check assumptions
Plan ignored while executing	Diff doesn't match	Execute in steps, and compare
Tests pass, and behavior's wrong	Weak tests	Review tests, and include edge cases
Forgot after compaction	Repeats questions	Write down decisions
Stale resume	Refers to old code	Tell it, or start fresh
Keeps correcting the same thing	Two failed rounds	Clear, and write a better prompt

Java

There's no Claude-specific code in this lesson. What matters in a Java repo is a one-line, fast check that it can run, in CLAUDE.md:

```
# Commands
- One test class: mvn -B test -Dtest=OrderServiceTest < /dev/null
- All tests: mvn -B test < /dev/null
```

And a concrete example beats prose when asking for a transformation:

```
Convert amounts to minor units:
  "12.50 EUR" -> {"minor": 1250, "currency": "EUR"}
  "1,000 JPY" -> {"minor": 1000, "currency": "JPY"}
  "" -> null
```

The closed stdin on the Maven lines comes from this course's experience (AI08). It stopped repeated hangs here, and I can't say that it's the only cause.

Trade-offs

Decision	Benefit	Cost
Plan mode	Avoids rework	Takes time up front
Direct	Fast	Costly if wrong
Tests first	Clear signal	Writing them first
Interview	Finds unknowns	More questions
One big message	Handles interaction	Harder to attribute changes
New session with summary	Fresh and accurate	Loses details

When not to plan: if you could describe the diff in one sentence, skip it. That rule of thumb is my own.

Common mistakes

Staying in the same session across unrelated tasks. Old context keeps competing for attention. Use `/clear`.

Common mistakes

Saying "it's wrong" without saying how. Give the failing test, or an expected output.

What to say in an interview

"I plan when the approach is uncertain or being wrong would be costly, using Explore for discovery, and then execute the plan directly. A clear single-file fix goes straight to execution. I steer with evidence: a few input/output examples, failing tests, or an interview in unfamiliar areas. Interacting issues go in one message. And I give it a pass-or-fail check, because it otherwise stops when things only look done."

Certification lens

How this lesson maps to CCAR-F

TS 3.4 and TS 3.5 are exam content, and sessions are TS 1.7. Know the plan-versus-direct criteria, Explore, the combination of plan then execute, 2-3 examples, test-driven iteration, the interview pattern, and when to send issues together. `/goal` and `/branch` aren't in the guide.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Code Generation · Easy. A stack trace points to a missing null check at one line in one file. How should you proceed?

A. Direct execution

B. Plan mode with an architecture review

C. Explore subagent, then a plan

D. Fork the session to compare two fixes

A **A** (TS 3.4). It's clear and small. The others are disproportionate.

Q Q2 · Exam style, scenario: Code Generation · Medium. You're choosing between two event-bus options with different infrastructure needs, across many services. What's best?

A. Implement the first option, and change it later if needed

B. Ask for both, in one branch

C. Plan mode, to explore the code and compare the options before editing

D. A one-line request to choose the best option and implement it

A **C** (TS 3.4). Multiple valid approaches with architectural effects call for planning. The others risk costly rework.

Q Q3 · Exam style, scenario: Code Generation · Medium. Claude's date converter gives different results from your prose description on each try. What's the most effective next step?

A. Rewrite your description in stronger terms

B. Ask it to reason more carefully

C. Lower the output token limit

D. Give 2-3 concrete input and output examples

A **D** (TS 3.5). Examples communicate a transformation best. The others don't remove the ambiguity.

Q Q4 · Exam style, scenario: Code Generation · Medium. You're adding caching in an area new to you, and you're unsure what could go wrong. What technique fits?

A. Immediate implementation, then fixing bugs as they come

B. The interview pattern, so that Claude asks about invalidation, failure modes and so on first

C. Asking for the most popular caching library's defaults

D. A skill with `context: fork` to implement it in isolation

A **B** (TS 3.5). It surfaces what you may have missed. The others skip the questions you don't yet know to ask.

Q Q5 · You resume a session after a rebase that changed most files it read. What should you do, and why?

A Start a new session with a structured summary of decisions and remaining work (TS 1.7). Its stored tool results describe old code, and it'll reason from them confidently. Resuming and naming files suits a few changes. Here, most are stale.

Revision - Lesson 4

→ Plan for large, uncertain or architectural work. Direct for clear, small changes.

→ Plan mode reads and plans, with no source edits. Then execute the plan.

→ Explore isolates verbose discovery, and returns a summary.

→ 2-3 input/output examples beat prose. Share failing tests. Interview in unfamiliar areas.

→ Interacting issues go in one message, and independent ones in sequence.

→ Give Claude a pass-or-fail check, as it stops when work looks done.

→ `--resume <name>`, `--continue`, `--fork-session`, `/branch`, `/clear`, `/compact`.

→ Resume and name changes when context is mostly valid. Start fresh when it's mostly stale.

→ Forks branch the conversation and not the files.

Your notes